

Lecture 3: The Job Scheduling Problem

(Greedy Algorithm - 01)

Date: August 10, 2026
Instructor: Aniket Basu Roy
Scribe: Siddhant Profulla Datta, Antra Gupta

Overview. Every problem we study this semester will be treated in the same four steps: *formalize* the problem precisely, *design* an algorithm, *prove* the algorithm correct, and *analyze* its efficiency. Today we carry out all four steps in full on our first greedy algorithm, the JOB SCHEDULING PROBLEM.

1 The Job Scheduling Problem

Motivation. Consider a set of jobs, where each job has a starting time and an ending time. We want to schedule the jobs on a single processor so that the maximum number of jobs are completed before their respective ending times. Since only one processor is available, no two jobs can overlap or be executed simultaneously until the job running is executed completely. The goal is to select a set of non-overlapping jobs that maximizes the number of jobs completed on time.

Definition 1 (Instance). An instance consists of a set $J = \{J_1, \dots, J_n\}$ of jobs, where each job J_i occupies a closed time interval $[s_i, e_i]$, with a starting time s_i and an ending time e_i , with $s_i < e_i$.

Definition 2 (Overlapping). Two jobs J_m, J_n are said to be overlapping when starting time of J_m is less than finishing time of J_n .

Definition 3 (Feasible schedule). A feasible schedule S is a subset of jobs $S \subseteq J$ such that no two jobs in S overlap in time; that is, for every two distinct jobs $J_i, J_k \in S$, their intervals $[s_i, e_i]$ and $[s_k, e_k]$ are disjoint.

Definition 4 (Feasibility). Given a schedule S , a pair of jobs $J_i, J_k \in S$ is an *overlap* if their time intervals $[s_i, e_i]$ and $[s_k, e_k]$ intersect. A schedule with no overlapping jobs is *feasible*.

Worked example. With $J = \{J_1, J_2, J_3, J_4\}$ and job intervals $J_1 = [1, 4]$, $J_2 = [3, 5]$, $J_3 = [5, 7]$, and $J_4 = [6, 8]$, the schedule $\{J_1, J_3\}$ is *feasible*, since the two jobs do not overlap. The schedule $\{J_1, J_2\}$ is *infeasible*, since J_1 and J_2 overlap during the interval $[3, 4]$. Similarly, $\{J_2, J_3\}$ is infeasible if the intervals are closed, since they both include time 5.

Thus, an arbitrary subset of jobs need not form a feasible schedule — an algorithm is needed to select a maximum-size set of mutually non-overlapping jobs.

2 The Finish First Algorithm

Algorithm 1: Finish First

Input: A set of jobs $J = \{J_1, \dots, J_n\}$ with time intervals $[s_i, e_i]$ for each job J_i
Output: A maximum-size set S of mutually non-overlapping jobs

```

1  $S \leftarrow \emptyset$ 
2 Sort all jobs in  $J$  in increasing order of ending times  $e_i$ 
3  $Endtime \leftarrow zero$ 
4 foreach job  $J_i$  in sorted order do
5   if  $s_i > Endtime$  then
6     Add  $J_i$  to  $S$ 
7      $Endtime \leftarrow e_i$ 
8 return  $S$ 
```

Algorithm 1: The intuition is that the job intervals are arranged in increasing order of their finishing time. The algorithm picks the job that finished first and remove the job intervals that overlap with this interval, and repeat till the set of all the jobs is empty. There is a set of jobs, an empty set S and there is a variable $Endtime$ whose initial value is 0, the jobs are sorted in increasing order of their ending time. The algorithm picks the job that has smallest ending time add it to the set S , set the variable $Endtime =$ end time of the job picked, then it picks

the next job from the sorted list if its starting time $>$ value of Endtime then add it to the set S otherwise discard the job. Repeat this process until the set of all jobs is empty. Return S , that will be the optimum solution

3 Correctness

Let $\{I_1, I_2, \dots, I_k\}$ be the set of jobs in S be the algorithm schedule in the order they were added to S . Note that $|S| = k$. Similarly, let the set of jobs in O be the optimal schedule denoted by $\{J_1, \dots, J_m\}$. The intuition for the greedy method came from wanting the resource to become free again as soon as possible after completing the first job. And greedy rule guarantees that. We have to prove that for each $r \geq 1$, the r^{th} accepted job in the algorithm's schedule $f(I_r)$ finishes no later than the r^{th} request in the optimal schedule $f(J_r)$ i.e.

$$f(I_r) \leq f(J_r)$$

Claim 1. For all indices $r \leq k$, we have $f(I_r) \leq f(J_r)$.

Proof. By Induction: For $r = 1$ the statement is true: the algorithm starts by selecting the job with minimum finish time, so $f(I_1) \leq f(J_1)$. Since O consists of compatible intervals that $f(J_{r-1}) \leq s(J_r)$. Combining this with the induction hypothesis $f(I_{r-1}) \leq f(J_{r-1})$, we get $f(I_{r-1}) \leq s(J_r)$. Thus the interval J_r is in the set of available intervals (jobs) at the time when the greedy algorithm selects I_r . The greedy algorithm selects the available interval with smallest finish time; since interval J_r is one of these available intervals, we have $f(I_r) \leq f(J_r)$. This completes the induction step.

Claim 2. The greedy algorithm returns an optimum set S containing $\{I_1, I_2, \dots, I_k\}$

Proof. By Contradiction: Suppose the claim made is false then an optimal set O contains more jobs than S . Let $S = \{I_1, I_2, \dots, I_k\}$ and $O = \{J_1, J_2, \dots, J_m\}$, where $m > k$, since $m > k$, there exist atleast one job J_{k+1} in set O . This job starts after job J_k ends and hence after I_k ends i.e.

$$s(J_{k+1}) \geq f(J_k) \geq f(I_k)$$

So after deleting all the jobs that are not compatible with jobs $\{I_1, I_2, \dots, I_k\}$ the set of possible job still contains job J_{k+1} . But the greedy algorithm stops with job I_k , and it is only suppose to stop when there is no remaining job compatible with the jobs already selected, hence it is a Contradiction.

For $m < k$, is impossible as O is the optimal set. ■

4 Running Time

This algorithm runs in $O(n \log n)$. The algorithm begins with sorting the jobs according to their finishing time, this will take $O(n \log n)$ time. it now selects the job which has smallest finishing time it then iterates through the list (sorted) of the jobs until it finds the next job whose starting time finishing time of the previously selected job, spending constant time per interval. Thus this part of the algorithm takes $O(n)$ time. ■

5 Maximum size independent set

Intuition: Pick the interval which overlaps with minimum no of job intervals and ignore the jobs whose interval overlaps with the picked interval. We can think of it as a graph where, for each job there is a vertex and there is an edge between two vertices if the intervals overlap. Now just pick the vertex with the smallest degree, remove its neighbors and repeat. Hence, the problem boils down to finding the maximum size independent set from the graph. ■

Reference. Kleinberg, J., & Tardos, E. (2006). *Algorithm Design*, Chapter 4. Pearson/Addison-Wesley.